线段树区间修改

人员

于子珈、周子一、陈洛冉、赵熙羽、牛同泽、秦显森、谢亚锴、杨瑾硕、司云心 到课, 李子瀚 线上

上周作业检查

上周作业链接: https://cppoj.kids123code.com/contest/3731

王向东老师周日十点半C++线段树

刷新

#	用户名	姓名	编程分	时间	A	B	C
1	zhouziyi	周子一	300	1759	100	100	100
2	yangjinshuo	杨谨硕	200	489	100	100	
3	yuzijia1	于子珈	200	1461	100	100	
4	qinxiansen	秦显森	200	2022	100	100	
5	lizihan	李子瀚	200	2032	100	100	
6	niutongze	牛同泽	200	2044	100	100	
7	xieyakai	谢亚锴	200	2168	100	100	
8	zhaoxiyu	赵熙羽	200	2180	100	100	
9	chenluoran	陈洛冉	100	1331	100		
10	siyunxin	司云心	0	0		0	

本周作业

https://cppoj.kids123code.com/contest/3841 (课上讲了 A ~ B 题, 课后作业是 B 题必做)

课堂表现

今天的第二题会比较复杂一些, 需要同时维护 乘 和 加 两个懒标记, 大家写起来吃力比较正常, 今天的重点是同学们熟练掌握 A 题即可。

课堂内容

【模板】线段树 1

线段树区间修改模板题, 用一个懒标签加速区间修改操作, 额外维护一个 pushdown 操作

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

const int maxn = 1e5 + 5;
struct node {
  int l, r, sum, add;
  int len;
```

```
} tr[maxn*4];
int w[maxn];

void pushup(int u) { tr[u].sum = tr[u*2].sum + tr[u*2+1].sum; }

void pushdown(int u) {
  if (tr[u].add) {
    tr[u*2].add += tr[u].add, tr[u*2].sum += tr[u].add * tr[u*2].len;
    tr[u*2+1].add += tr[u].add, tr[u*2+1].sum += tr[u].add * tr[u*2+1].len;
    tr[u].add = 0;
  }
}

void build(int u, int l, int r) {
  tr[u] = {l, r, 0, 0, r-l+1};
  if (l == r) { tr[u].sum = w[l]; return; }
  int mid = (l + r) / 2;
  build(u*2, l, mid), build(u*2+1, mid+1, r);
  pushup(u);
}

void modify(int u, int l, int r, int k) {
  if (tr[u].l>=l && tr[u].r<=r) {
    tr[u].add += k, tr[u].sum += k * tr[u].len; return;
  }

  pushdown(u);
  int mid = (tr[u].l + tr[u].r) / 2;
  if (l <= mid) modify(u*2, l, r, k);
  if (r > mid) modify(u*2+1, l, r, k);
  pushup(u);
}

int query(int u, int l, int r) {
  if (tr[u].l>=l && tr[u].r<=r) return tr[u].sum;

  pushdown(u);
  int mid = (tr[u].l + tr[u].r) / 2, res = 0;
  if (l <= mid) res += query(u*2, l, r);
  if (r > mid) res += query(u*2+1, l, r);
  return res;
}

signed main()
{
  int n, m; cin >> n >> m;
  for (int i = 1; i <= n; ++i) cin >> w[i];
  build(1, 1, n);

  while (m -- ) {
    int op, l, r; cin >> op >> l >> r;
    if (op == 1) {
      int k; cin >> k; modify(1, l, r, k);
    } else cout << query(1, l, r) << endl;
```

```
    }
    return 0;
}
```

【模板】线段树 2

稍微复杂的线段树操作, 需要维护 2 个懒标签, 分别代表区间整体乘了多少, 以及区间整体加了多少

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

const int maxn = 1e5 + 5;
struct node {
    int l, r, add, mul, sum, len;
} tr[maxn*4];
int w[maxn];
int mod;

void pushup(int u) {
    tr[u].sum = tr[u*2].sum + tr[u*2+1].sum; tr[u].sum %= mod;
}
void pushdown(int u) {
    node &uu = tr[u], &ll = tr[u*2], &rr = tr[u*2+1];
    ll.mul *= uu.mul, ll.mul %= mod;
    ll.add = ll.add*uu.mul + uu.add, ll.add %= mod;
    ll.sum = ll.sum*uu.mul + uu.add*ll.len, ll.sum %= mod;

    rr.mul *= uu.mul, rr.mul %= mod;
    rr.add = rr.add*uu.mul + uu.add, rr.add %= mod;
    rr.sum = rr.sum*uu.mul + uu.add*rr.len, rr.sum %= mod;
    uu.add = 0, uu.mul = 1;
}
void build(int u, int l, int r) {
    tr[u] = {l, r, 0, 1, 0, r-l+1};
    if (l == r) {
        tr[u].sum = w[l]; return;
    }
    int mid = (l + r) / 2;
    build(u*2, l, mid), build(u*2+1, mid+1, r);
    pushup(u);
}

void modify1(int u, int l, int r, int k) {
    if (tr[u].l>=l && tr[u].r<=r) {
        tr[u].mul *= k, tr[u].add *= k, tr[u].sum *= k;
        tr[u].mul %= mod, tr[u].add %= mod, tr[u].sum %= mod;
        return;
    }
```

```
    pushdown(u);
    int mid = (tr[u].l + tr[u].r) / 2;
    if (l <= mid) modify1(u*2, l, r, k);
    if (r > mid) modify1(u*2+1, l, r, k);
    pushup(u);
}

void modify2(int u, int l, int r, int k) {
    if (tr[u].l>=l && tr[u].r<=r) {
        tr[u].add += k, tr[u].sum += k * tr[u].len;
        tr[u].add %= mod, tr[u].sum %= mod;
        return;
    }

    pushdown(u);
    int mid = (tr[u].l + tr[u].r) / 2;
    if (l <= mid) modify2(u*2, l, r, k);
    if (r > mid) modify2(u*2+1, l, r, k);
    pushup(u);
}

int query(int u, int l, int r) {
    if (tr[u].l>=l && tr[u].r<=r) return tr[u].sum;

    pushdown(u);
    int mid = (tr[u].l + tr[u].r) / 2, res = 0;
    if (l <= mid) res += query(u*2, l, r);
    if (r > mid) res += query(u*2+1, l, r);
    return res % mod;
}

signed main()
{
    int n, m; cin >> n >> m >> mod;
    for (int i = 1; i <= n; ++i) cin >> w[i];
    build(1, 1, n);
    while (m -- ) {
        int op, x, y, k; cin >> op >> x >> y;
        if (op == 1) {
            cin >> k;
            modify1(1, x, y, k);
        } else if (op == 2) {
            cin >> k;
            modify2(1, x, y, k);
        } else {
            cout << query(1, x, y) << endl;
        }
    }
  return 0;
}
```